

Reverse Engineering と CodeKB

既存コードを読んで、チームで共有できるコード理解のストアを作る仕組み

AI-DLC ドキュメント · User Guide · フェーズとステージ · 成果物リファレンス

Reverse Engineering は、既存コードを読んで **CodeKB**(コード知識ベース)を作るステージです。下流の要件・設計・実装が、いまのコードベースを同じ理解から読めるようにします。

人が Markdown で書く 第2層ナレッジ(「いつもこう作る」)とは別です。既存文書を ID で引用する DocumentKB とも別です。

外部の CodeKB MCP(任意の構造分析サーバ)とも別です。この章の CodeKB は、フレームワークが書くローカルストア `aidlc/spaces/<space>/codekb/` です。

目次

1. 何か
2. どのように進むか
3. いつ走るか
4. 人が選ぶこと(再利用と再スキャン)
5. 生成される成果物
6. 他のナレッジとの使い分け
7. 次に読む

何か

エージェントは、リポジトリを見ただけでは、ドメイン・構成・API・依存をチームで共有できる形にはしません。Reverse Engineering が走って初めて、次が決まります。

- 何のシステムか(ビジネス概要、アーキテクチャ)
- コードがどう分かれているか(構成、コンポーネント、技術スタック)
- どこまで深く読んだか(鮮度と被覆。次のIntentが再利用できるか)

成果物はIntentのレコードではなく、**スペース共有・リポジトリごと**の店に置きます。同じスペースの後続Intentは、自分のスキャン時点ではなく、そのリポジトリの最新ストアを読みます。

```
aidlc/spaces/<space>/
├─ codekb/                # Reverse Engineering が書く。手で直さない
│   └─ <repo>/            # リポジトリにつき 1 ストア
│       ├── business-overview.md
│       ├── architecture.md
│       ├── code-structure.md
│       ├── api-documentation.md
│       ├── component-inventory.md
│       ├── technology-stack.md
│       ├── dependencies.md
│       ├── code-quality-assessment.md
│       └─ reverse-engineering-timestamp.md
└─ intents/<record>/      # このステージの日記と、スキャンの手渡しだけ
```

<space> は名前付きスペースを使わなければ **default** です。**<repo>** はインテント作成時に記録したリポジトリ名です。**repos** が無い単一リポジトリでは、ワークスペースルートに対して 1 回走ります。

codekb/ は git に載せます。チームが同じコード理解を共有するための設計です。公開はステージがロック付きで行うので、ファイルを手で置き換えしないでください。

どのように進むか

ステージ番号は **2.1**、実行は **パイプライン** (2 段の鎖) です。コンダクターが順に委譲し、人がスキャン中に割り込む必要はありません。

- 1 **Developer** (**aidlc-developer-agent**) がコードをスキャンする。パッケージ、ビルド、API、依存、テスト、品質の兆候、技術的負債を、手渡しファイルに書く
- 2 **Architect** (**aidlc-architect-agent**) がそれを合成し、下の 9 ファイルを書く。 **architecture.md** にはコンポーネント関係の Mermaid と、ビジネストランザクションの相互作用図が入ります

スキャンが終わると承認ゲートが出ます。Approve で Requirements Analysis へ進み、Request Changes で直しを求めます。

複数リポジトリのインテントでは、記録したリポジトリごとにこの鎖が 1 本完走します。ストアはリポジトリごとに独立です。

いつ走るか

次を満たすと、ワークフローの中でこのステージが計画に入ります。

条件	内容
プロジェクト種別	brownfield だけ。greenfield は Initialization(0.2 の検出)で自動 SKIP

条件	内容
スコープ	<code>infra</code> 以外。 <code>enterprise</code> 、 <code>feature</code> 、 <code>mvp</code> 、 <code>poc</code> 、 <code>bugfix</code> 、 <code>refactor</code> 、 <code>security-patch</code> 、 <code>classic</code> 、 <code>workshop</code> 、 <code>express</code> に含まれる
タイミング	通常は Ideation の承認ゲートのあと、Inception の先頭。 <code>bugfix</code> などは Ideation を飛ばすので、Initialization の直後
対象	Intent の <code>repos</code> 。未記録ならワークスペースルート 1 回

brownfield は、ステージ 0.2 がソース、フレームワーク設定、パッケージマニフェスト、`src/` などのソースディレクトリ、`.gitmodules` のいずれかを見つけたときです。トップに合図が無いときは、コンテナフォルダを最大 3 段まで見ます。ソースが `backend/` や `services/api/` に入っている場合でも brownfield になります。

計画に入っている場合でも、毎回フルスキャンするとは限りません。既存ストアの鮮度を見て、人が次を選びます。

ワークフロー全体を回さず、このステージだけ当てるときは次です。成果物は書き、ワークフローの **Current Stage** は進みません。

```
/aidlc --stage reverse-engineering --single
```

適応型コンポーザーが、外部の構造分析を根拠に SKIP を提案することがあります。黙っては飛ばしません。下流がローカル CodeKB 無しで走ることを開示し、決めるのはゲートの人です。

人が選ぶこと(再利用と再スキャン)

CodeKB はスペース単位で、Intent をまたいで共有します。再実行の直前に、ストアの `reverse-engineering-timestamp.md` を見て判定します。

判定	意味	人が選べること
<code>NO_STORE</code>	そのリポジトリの初回	質問なしでスキャン
<code>CURRENT</code> (被覆が足りる)	分析したパスは、ストアを作ったときから変わっていない	再利用 / フル再スキャン / 焦点スキャン
<code>CURRENT</code> (被覆が足りない)	ストアは新しいが、この Intent の領域を深くは読んでいない	フル再スキャン / 焦点スキャン
<code>STALE</code>	分析したパスの中身が変わった	フル再スキャン / 焦点スキャン
<code>UNVERIFIED</code>	指紋を計算できない(例: git 作業ツリーではない)	フル再スキャン / 焦点スキャン
<code>UNKNOWN_SCOPE</code>	スコープ追跡より前のストア	フル再スキャン / 焦点スキャン

選び方の意味:

- **再利用** — スキャンしない。下流はいまのストアをそのまま読む。記録したリポジトリをすべて再利用したとき、ステージは skipped
- **フル再スキャン** — リポジトリ全体を読み直し、9 ファイルを置き換える
- **焦点スキャン** — このIntentの領域を読んでストアへマージする。その外の既存の文は残す。検証できた深い被覆は和集合として積み上がる。古いか検証できない以前の深い被覆は、文としては残し、浅い扱いに落とす

焦点スキャンのあとに被覆が狭くなったときは、承認の前に警告が出ます。Approve は狭い被覆を受け入れること、Request Changes でスキャンを広げられます。

未初期化の git サブモジュールがあると、スキャンがコードを読めないので警告が出ます。`git submodule update --init --recursive` を先に走らせてください。

生成される成果物

永続成果物は **9 点** です。場所は `aidlc/spaces/<space>/codekb/<repo>/`。Minimal 深度でも 9 ファイルと必須セクションは揃います。短くするのは、同じ一覧をファイル間で繰り返さないことです。

ファイル	中身
<code>business-overview.md</code>	ビジネスドメイン、目的、主要機能
<code>architecture.md</code>	システム構成、パターン、コンポーネント関係 (Mermaid)。ビジネスランザクションがコンポーネントをどう通るかの相互作用図が必須
<code>code-structure.md</code>	パッケージ／モジュールの分け方、ファイル分類、コードパターン
<code>api-documentation.md</code>	外部・内部の API、エンドポイント、契約
<code>component-inventory.md</code>	コンポーネント一覧、責務、依存
<code>technology-stack.md</code>	言語、フレームワーク、ライブラリと版
<code>dependencies.md</code>	外部依存と、パッケージ間の内部依存
<code>code-quality-assessment.md</code>	テスト、lint、CI/CD、ドキュメントの質、技術的負債
<code>reverse-engineering-timestamp.md</code>	実施日時、あればコミットハッシュ。末尾の Scope of Analysis が、次の再実行ガードが読む鮮度と被覆

`reverse-engineering-timestamp.md` の Scope of Analysis は、読みたかった範囲ではなく、**実際に深く読んだ範囲** を記録します。`kind: full` はリポジトリ全体を深く読んだときだけです。それ以外は `kind: partial` です。

レコードディレクトリに残るもの (CodeKB ではない):

ファイル	役割
<code><record>/inception/reverse-engineering/developer-scan[<repo>].md</code>	Developer から Architect への手渡し。パイプラインの鎖の証拠
<code><record>/inception/reverse-engineering/memory.md</code>	ステージの観察日記。手では編集しない

公開の途中で使うステージングディレクトリは一時入力です。共有の `codekb/` へ直接書きません。ロック付きの公開が、ソースがスキャン後に変わったことや、並行する別インテントの公開と衝突することを止めます。

下流(Requirements Analysis、Domain Design、Functional Design、Code Generation など)は、この 9 点を入力にします。あるのは brownfield のときです。

他のナレッジとの使い分け

	第2層ナレッジ	DocumentKB	CodeKB(この章)
中身	チームが書いた標準・方針	すでに存在する文書	既存コードから合成した理解
場所	<code>aidlc/spaces/<space>/knowledge/</code> (自由形式)	<code>knowledge/documents/</code> と派生カタログ	<code>aidlc/spaces/<space>/codekb/<repo>/</code>
作り方	人が Markdown を置く	<code>onboard</code> / <code>sync</code>	Reverse Engineering ステージ
例	コーディング規約、API の形	この案件の PRD、契約、仕様 PDF	アーキテクチャ図、API 面、依存

「いつもこう作る」はナレッジ、「この文書が根拠である」は DocumentKB、「このリポジトリはこうなっている」は CodeKB です。

次に読む

- 最初のワークフロー — brownfield で Reverse Engineering がどこに出るか
- フェーズとステージ — Inception での位置とパイプライン
- スコープ・深度・テスト戦略 — スコープごとの所属。`infra` では走らない
- 成果物リファレンス — レコードディレクトリと `codekb/` の例外
- DocumentKB — 既存文書のカタログ
- CLI コマンド — `--single` と `codekb-path` / `codekb-scope-diff`
- 実例 — バグ修正の通しでの 9 成果物

